Heapsort

Sorting with a heap · Algorithms § 2.4

This video

- How heapsort is related to **selection sort**
- **Phase 1:** turn the array into a heap
- **Phase 2:** take out the largest item, over and over (the book calls this the **sortdown**)
- The code and its costs
- How heapsort compares with the other sorts we've seen

The idea behind heapsort

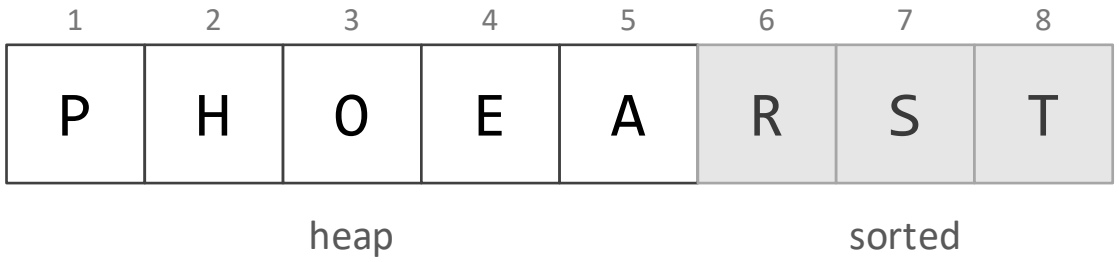
Selection sort: find the largest item that's left, and put it in its final place. Repeat.

(The book's version finds the smallest instead. Same idea.)

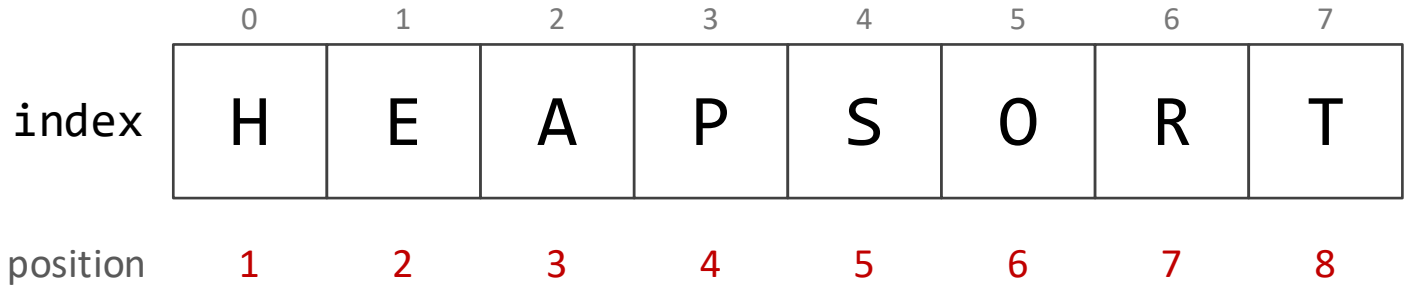
The slow part is **finding** it: a scan of everything that's left.

Heapsort: the same plan, but keep the items that are left in a **heap**. The largest is at the root, and removing it costs about **log n**.

We do all of this **inside the original array**. As the heap gets smaller, the sorted part grows from the right end.



Positions 1 to n, in an array that starts at 0



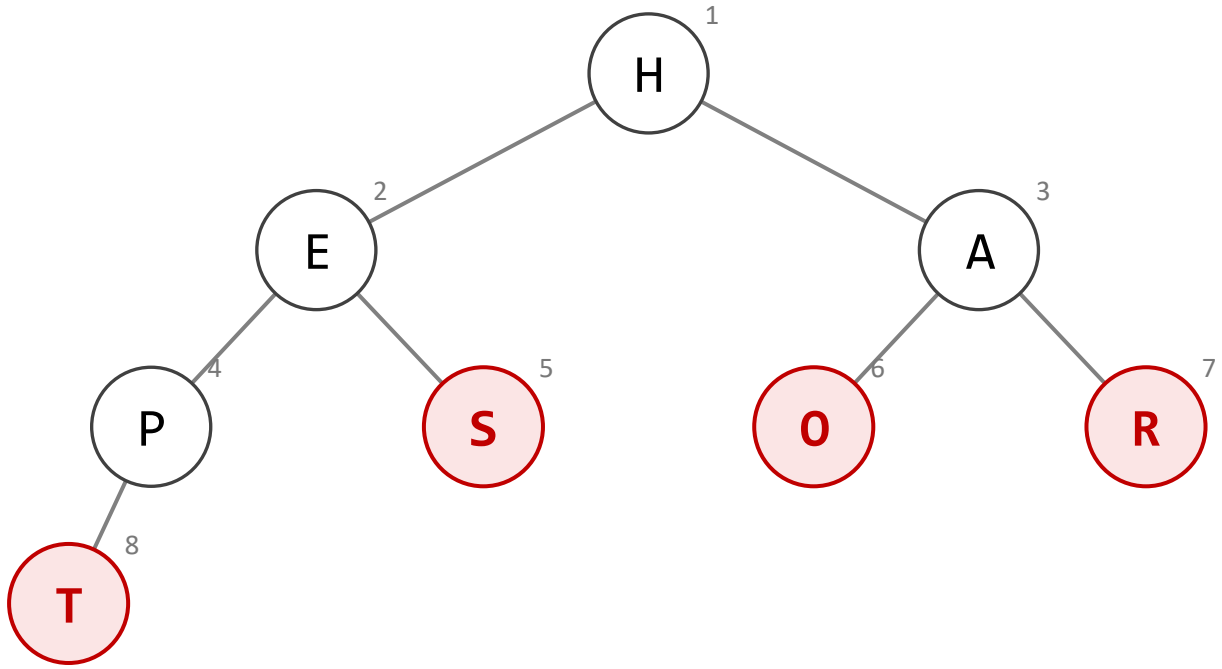
Heap arithmetic needs positions starting at **1** (children of k are 2k and 2k + 1).

The array starts at **0**. So heap position **k** is stored in `a[k - 1]`.

`less` and `exch` do the subtracting. Everything else, including `sink`, uses positions **1 to n**.

On the rest of the slides, the small numbers above the array are **positions**, not indices.

Phase 1: build a heap, from the bottom up



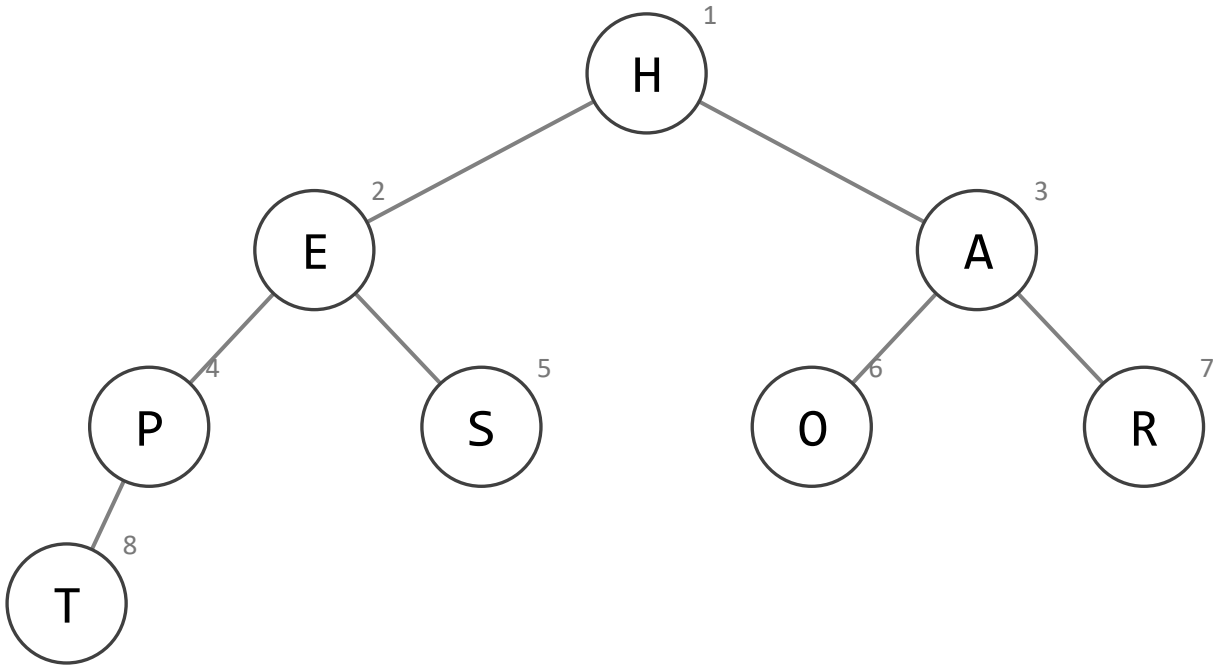

```
1  for (int k = n / 2; k >= 1; k--) {
2      sink(a, k, n);
3  }
```

Positions **5–8** have no children: each one is already a heap of one item (shaded).

Sink position **k = n / 2 = 4**, then **3**, then **2**, down to **1**.

Each sink joins a parent with the heaps below it into one bigger heap.

Phase 1: build a heap



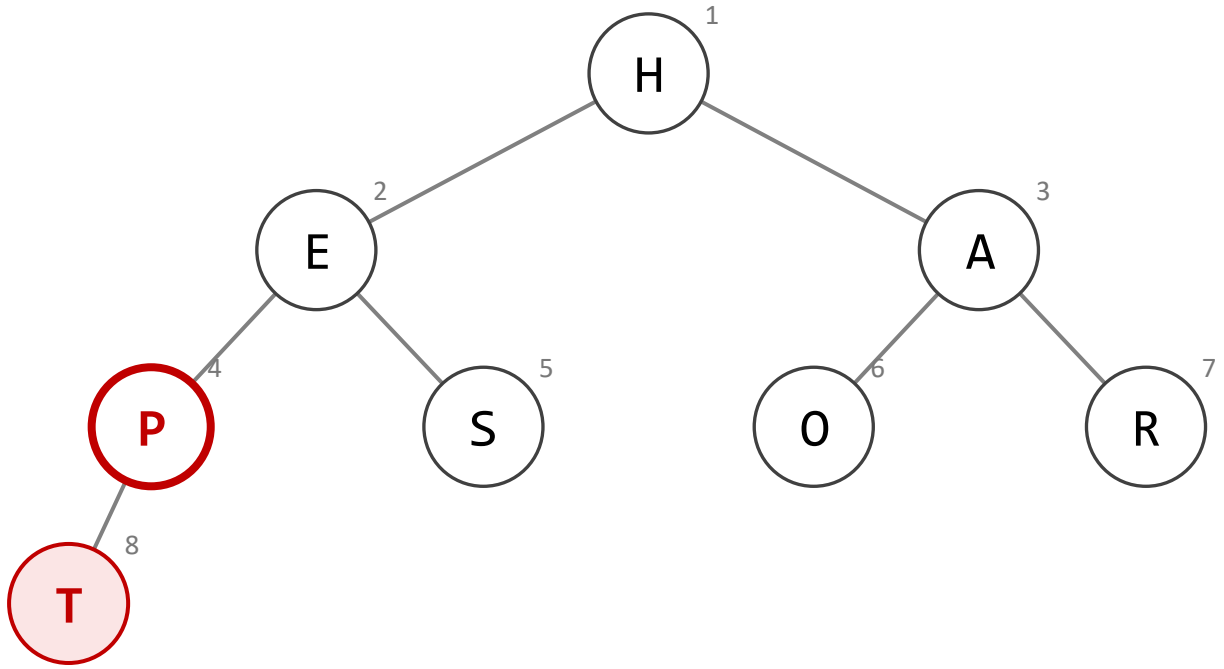
	1	2	3	4	5	6	7	8
a	H	E	A	P	S	O	R	T

Our array: **H E A P S O R T**, drawn as a tree.

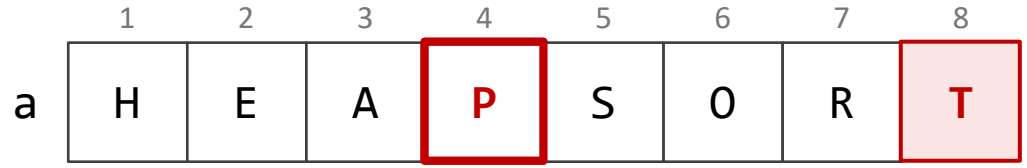
It's not a heap yet. Positions 5–8 are one-item heaps already.

Start with **k = 4**.

Phase 1: build a heap



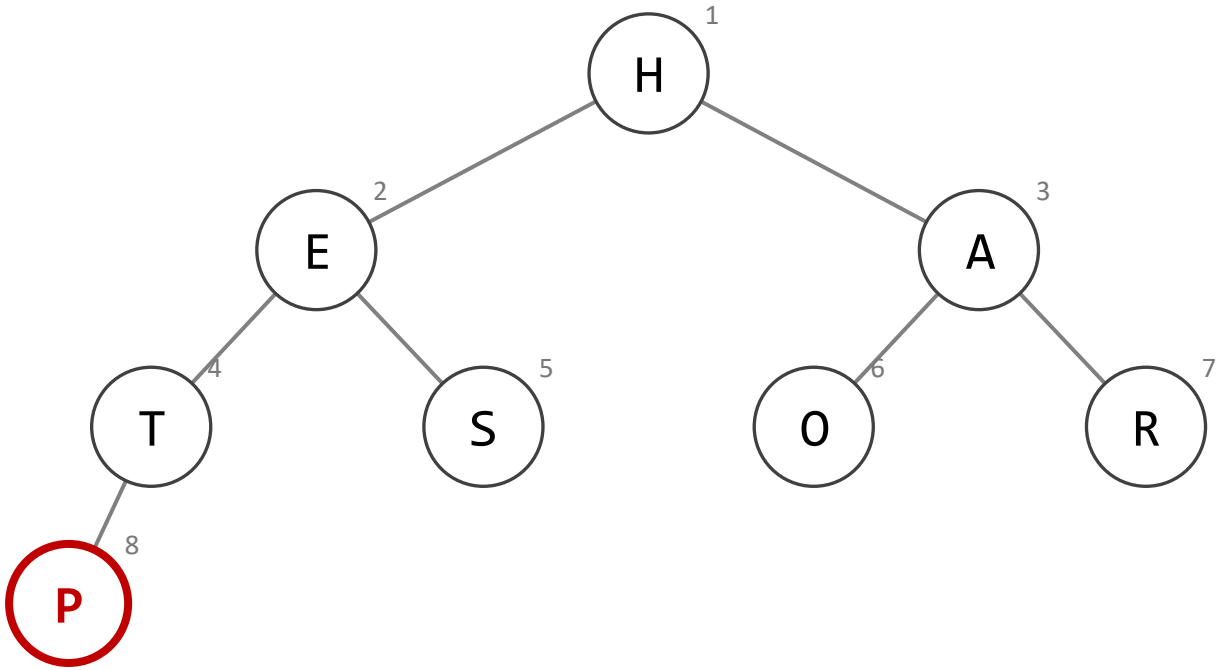
sink(a, 4, 8). P is at position 4. Its only child in the heap is position 8: T. Is P less than T? **Yes**: exchange them.



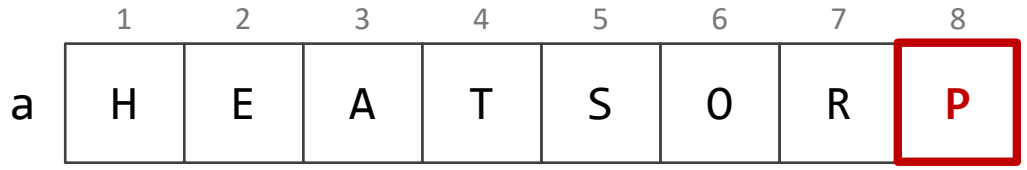
the item that's sinking

its larger child

Phase 1: build a heap



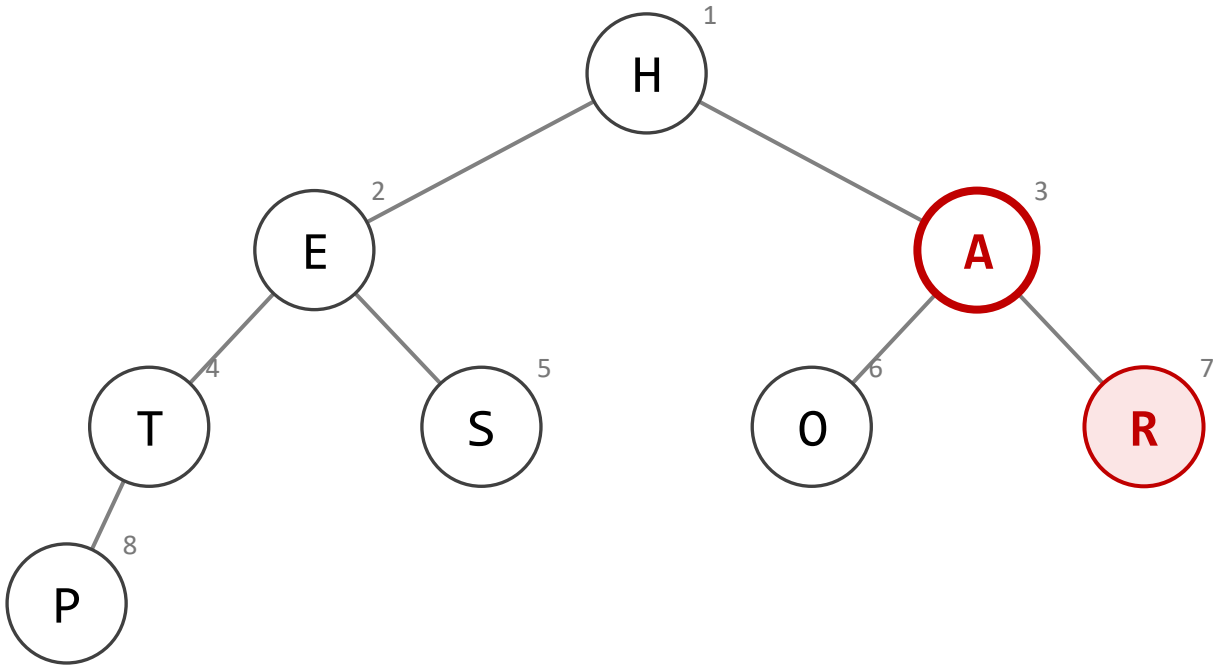
P is at position 8, which has no children in the heap (16 > 8). It stops.



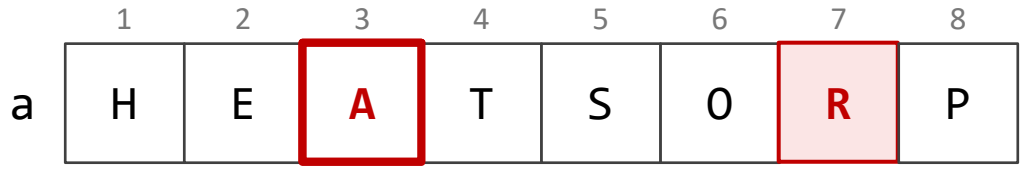
the item that's sinking

its larger child

Phase 1: build a heap



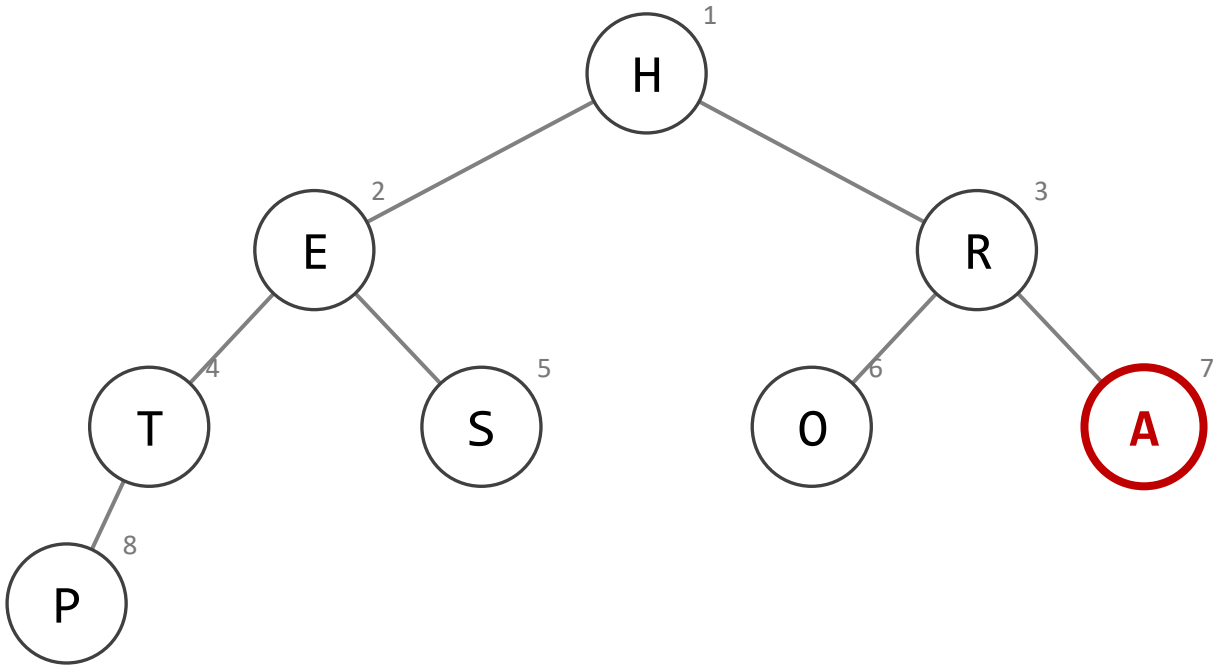
sink(a, 3, 8). A is at position 3. Its children are positions 6 and 7: O and R. The larger is **R**. Is A less than R? **Yes**: exchange them.



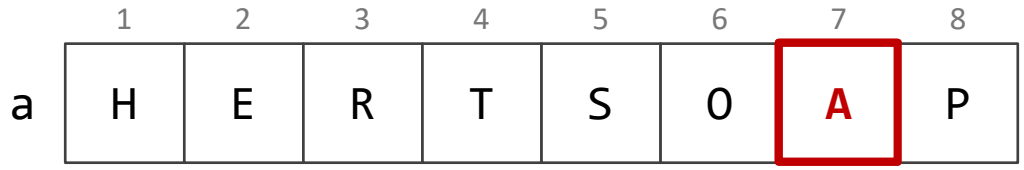
the item that's sinking

its larger child

Phase 1: build a heap



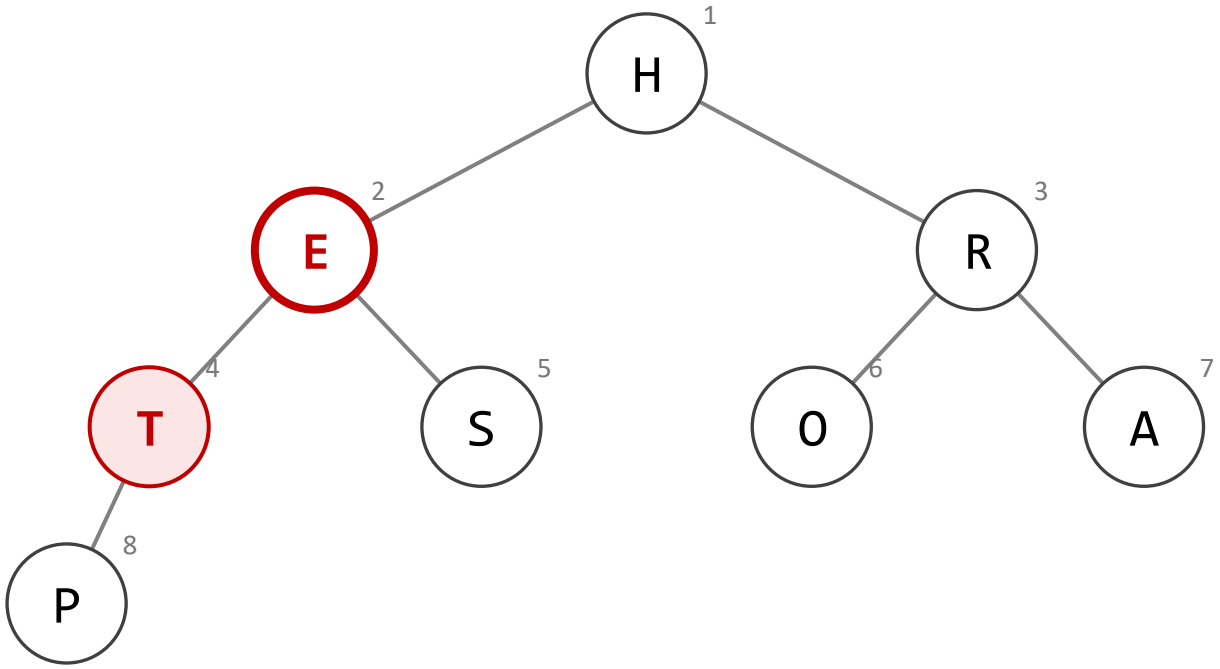
A is at position 7, which has no children in the heap (14 > 8). It stops.



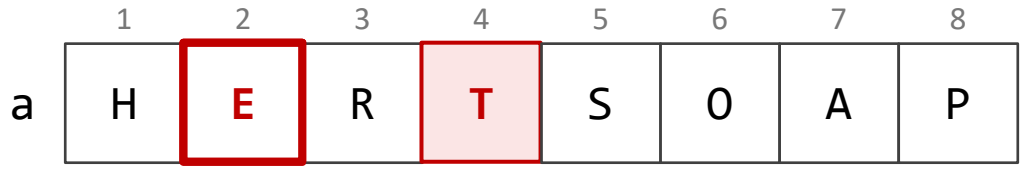
the item that's sinking

its larger child

Phase 1: build a heap



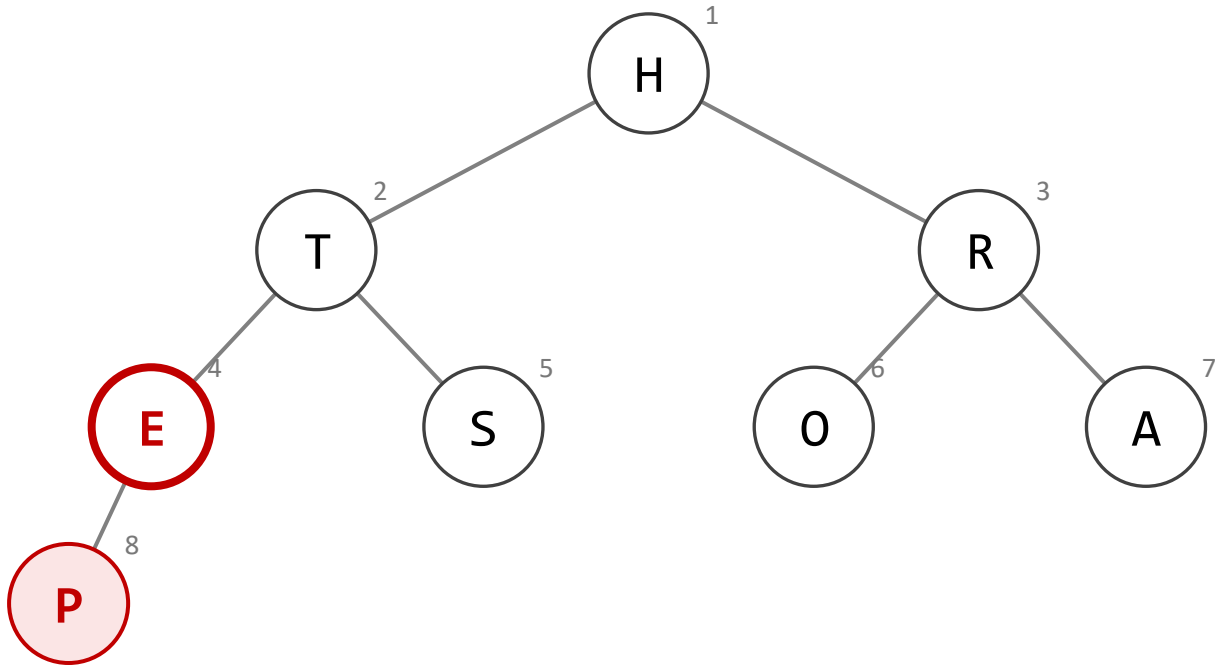
sink(a, 2, 8). E is at position 2. Its children are positions 4 and 5: T and S. The larger is **T**. Is E less than T? **Yes**: exchange them.



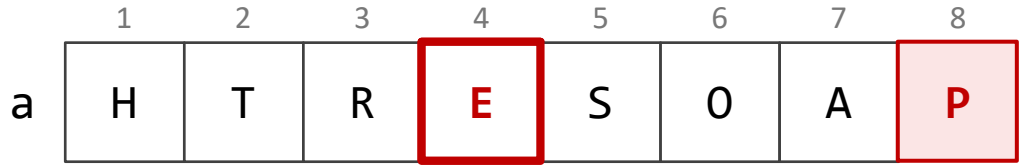
the item that's sinking

its larger child

Phase 1: build a heap



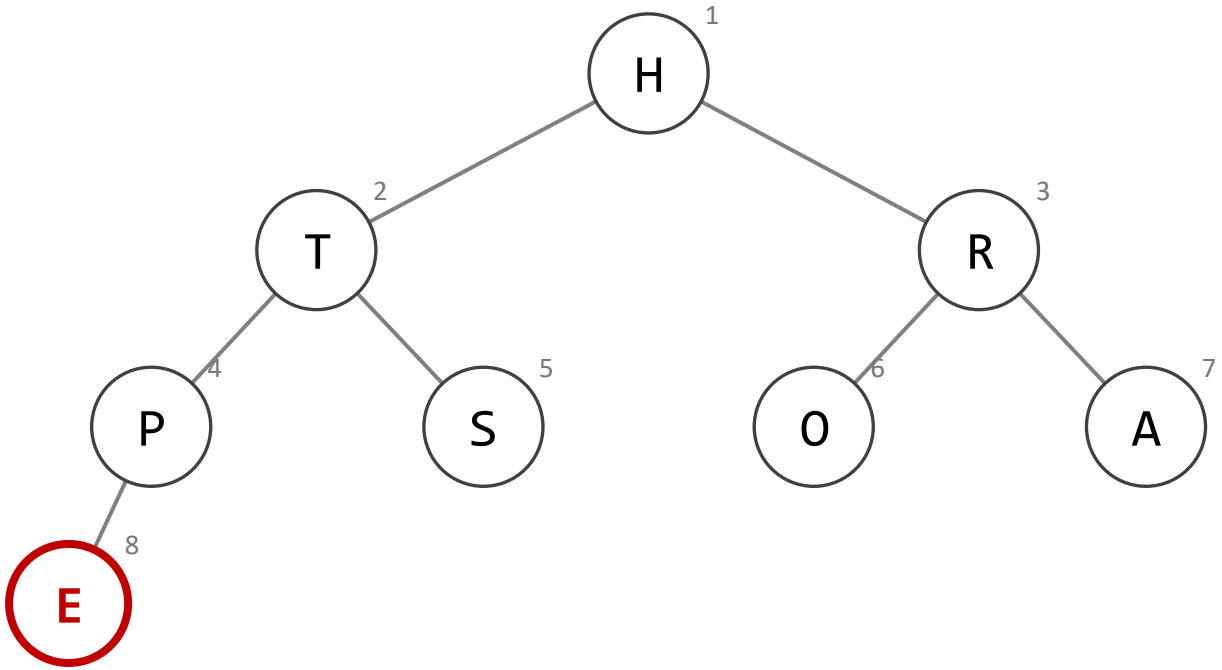
E is at position 4. Its only child in the heap is position 8: P. Is E less than P?
Yes: exchange them.



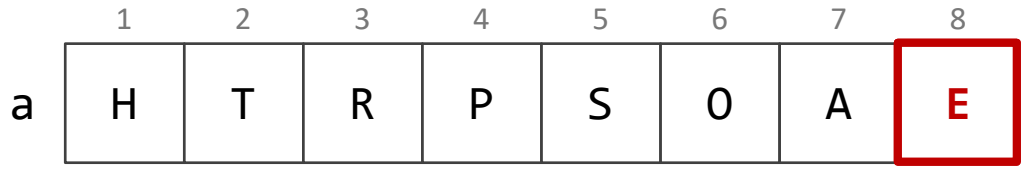
the item that's sinking

its larger child

Phase 1: build a heap



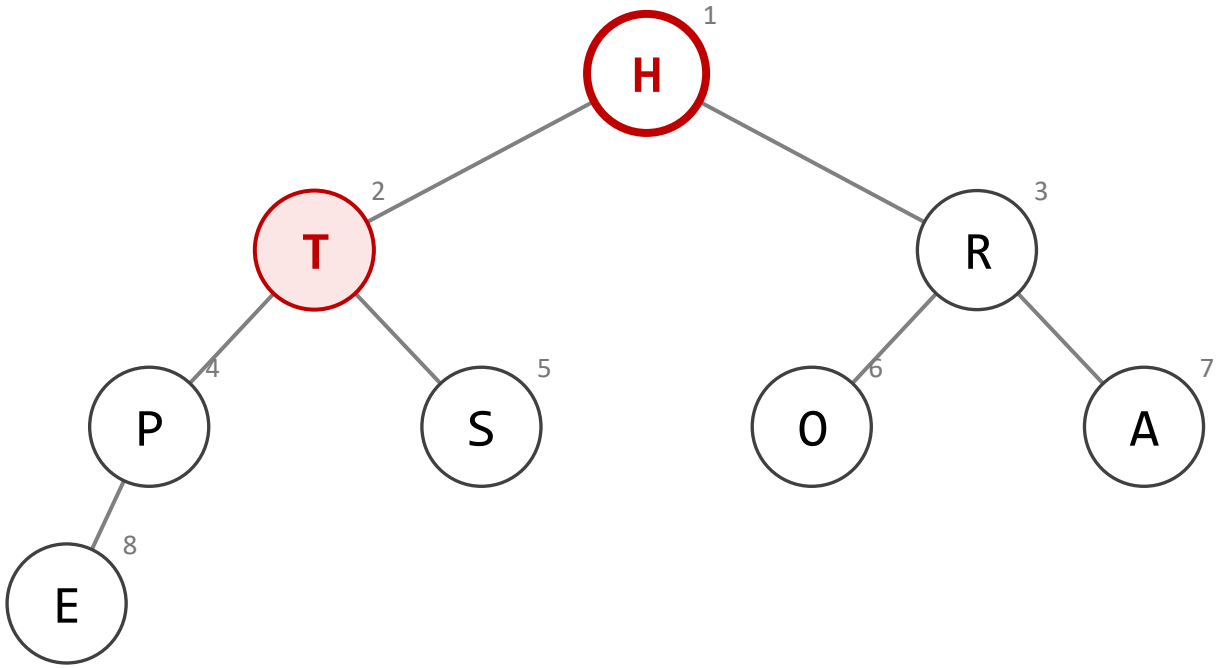
E is at position 8, which has no children in the heap (16 > 8). It stops.



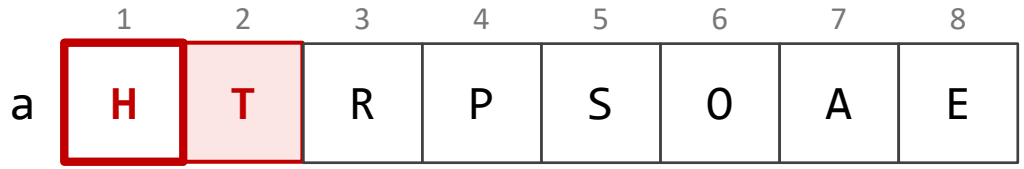
the item that's sinking

its larger child

Phase 1: build a heap



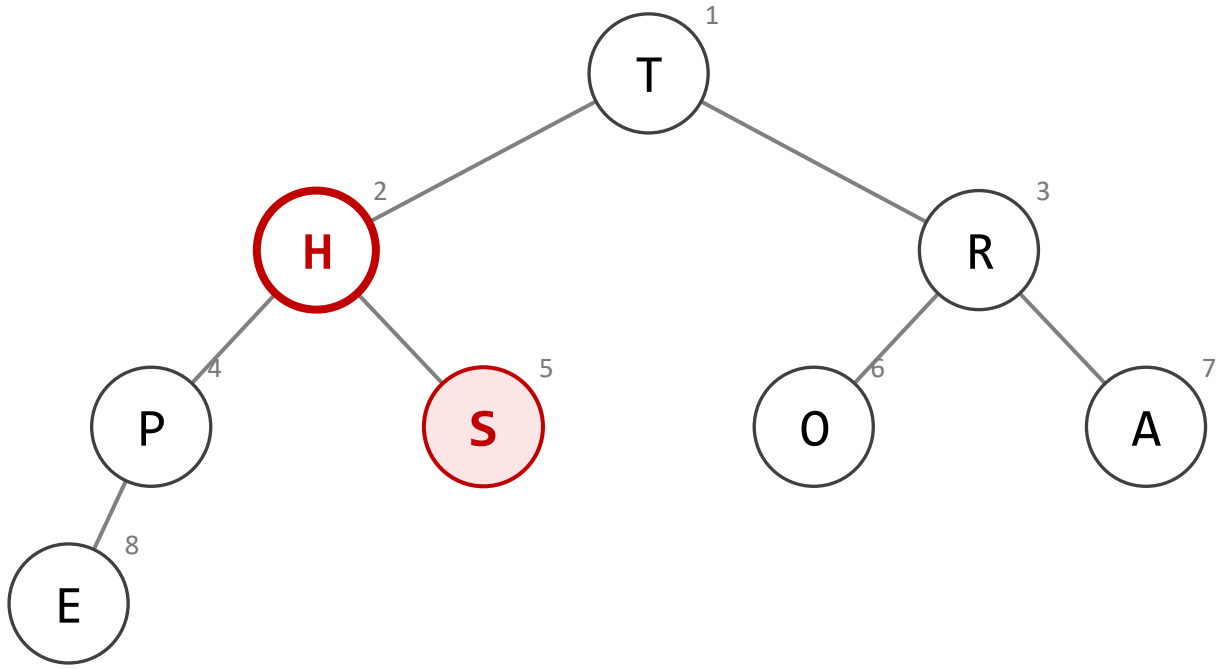
sink(a, 1, 8). H is at position 1. Its children are positions 2 and 3: T and R. The larger is **T**. Is H less than T? **Yes**: exchange them.



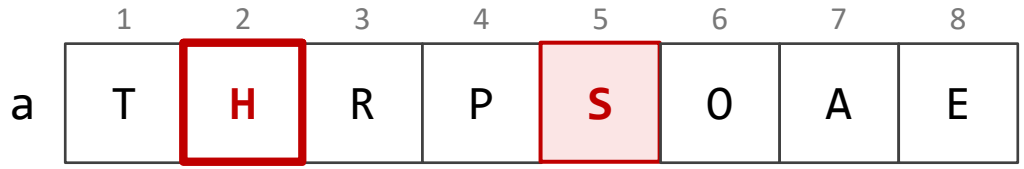
the item that's sinking

its larger child

Phase 1: build a heap



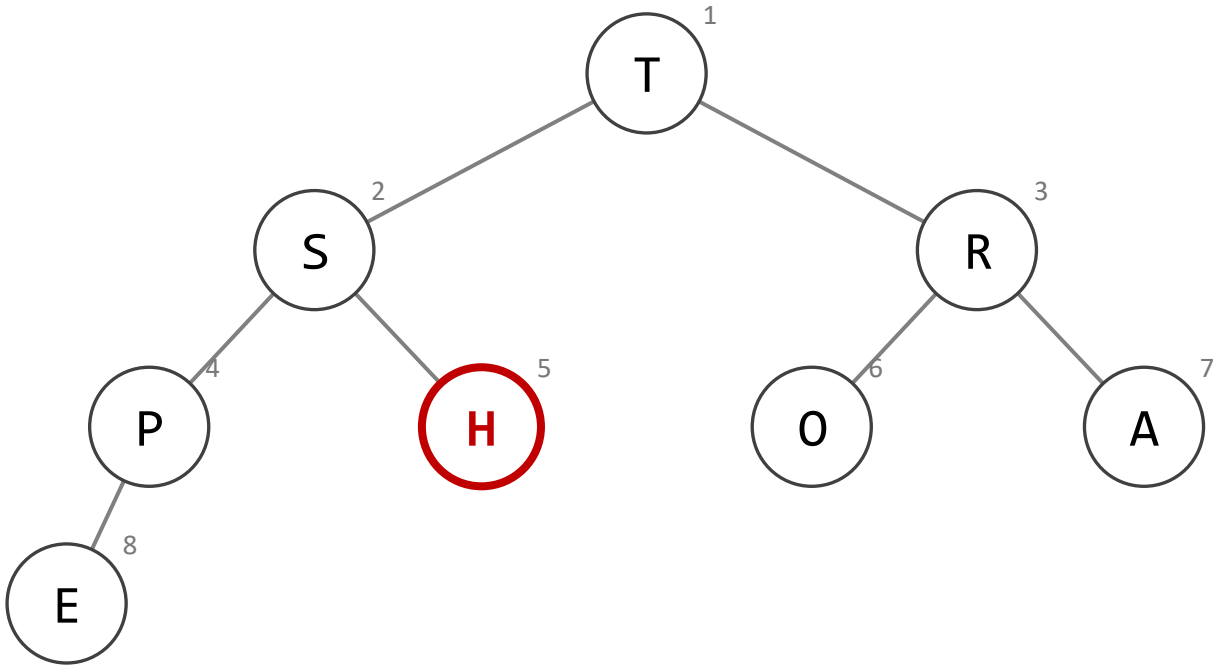
H is at position 2. Its children are positions 4 and 5: P and S. The larger is **S**. Is H less than S? **Yes**: exchange them.



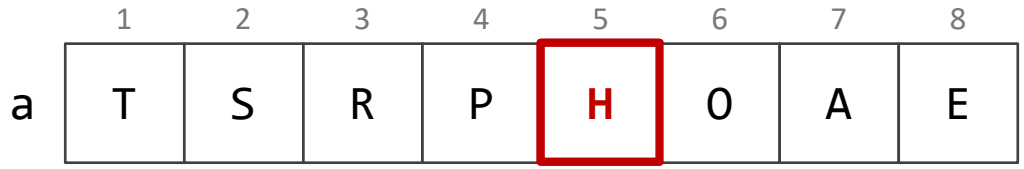
the item that's sinking

its larger child

Phase 1: build a heap



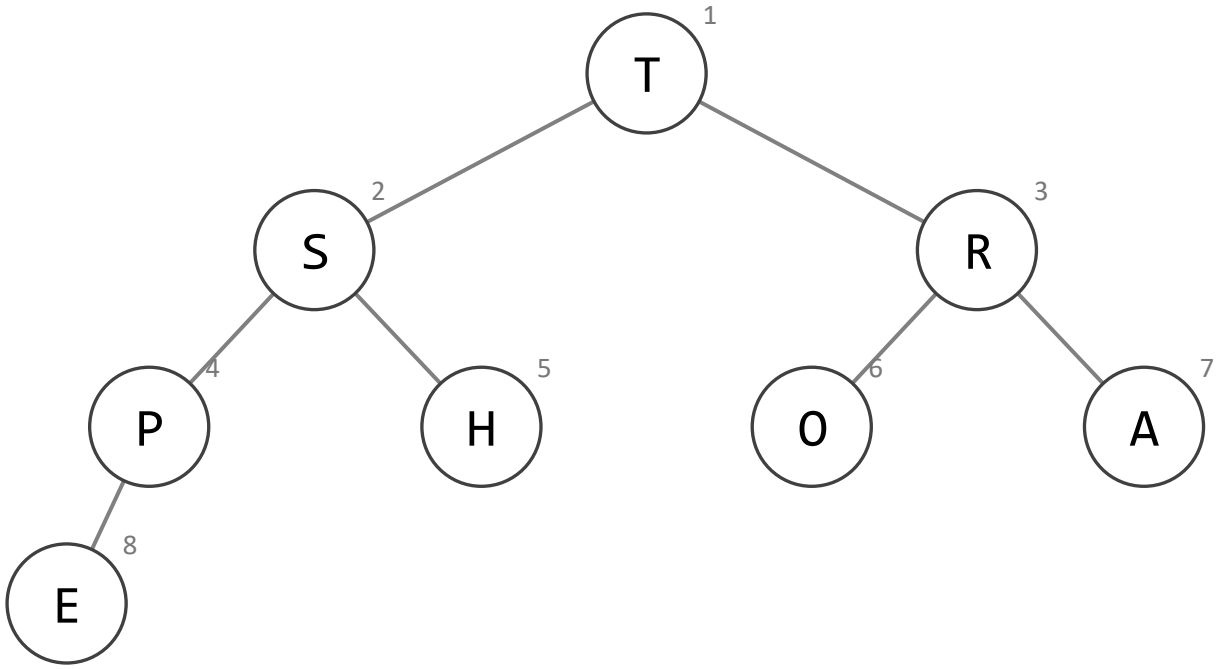
H is at position 5, which has no children in the heap (10 > 8). It stops.



the item that's sinking

its larger child

Phase 1: build a heap



	1	2	3	4	5	6	7	8
a	T	S	R	P	H	O	A	E

Phase 1 done. The array is a heap: **T S R P H O A E**.

The largest item, **T**, is at the root.

It took **6 exchanges**.

Why build from the bottom up?

Inserting the items one at a time: up to about **n lg n** compares.

Bottom-up: **fewer than 2n compares** and **fewer than n exchanges**. That's linear: the work grows in proportion to n.

Why? **Most items are near the bottom**, and they can only sink a short way.

- Half the items are leaves: they don't sink at all.
- A quarter can sink at most one level, an eighth at most two, and so on.

Phase 2: the sortdown

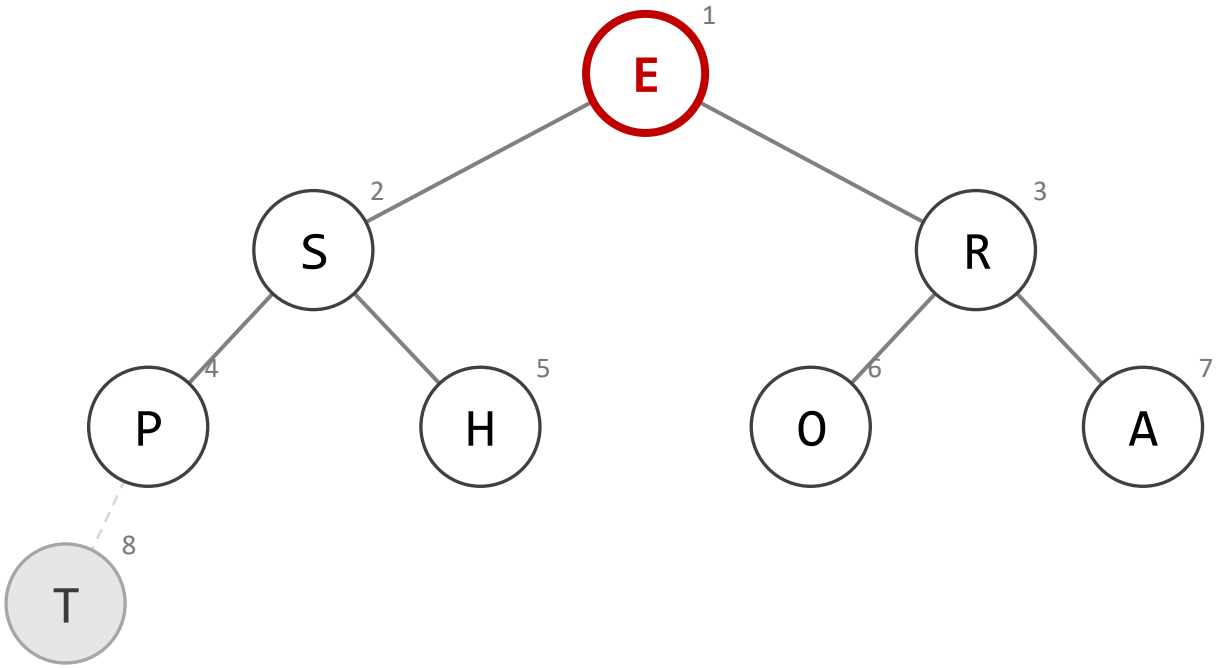
The book calls phase 2 the **sortdown**. It's **delMax** over and over, except we **keep** each max instead of returning it:

- **exch(a, 1, n):** the largest item goes to position n. That's its **final spot**.
- **n--:** it's no longer in the heap (gray).
- **sink(a, 1, n):** fix heap order from the new root.

Repeat until the heap has **one** item left.

```
1  while (n > 1) {
2      exch(a, 1, n);
3      n--;
4      sink(a, 1, n);
5  }
```

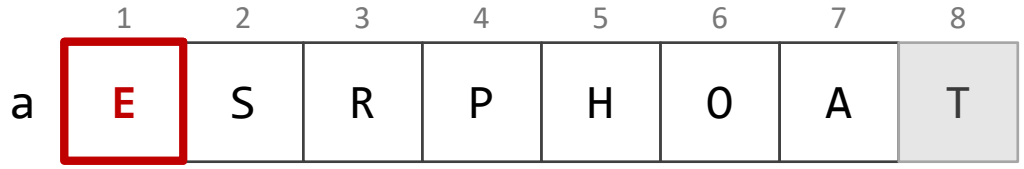
Phase 2: sortdown



exch(a, 1, 8): the largest, **T**, swaps with the last item in the heap, **E**.

n--: T is in its final spot (gray). The heap has **7** items.

Now sink **E** from the root.

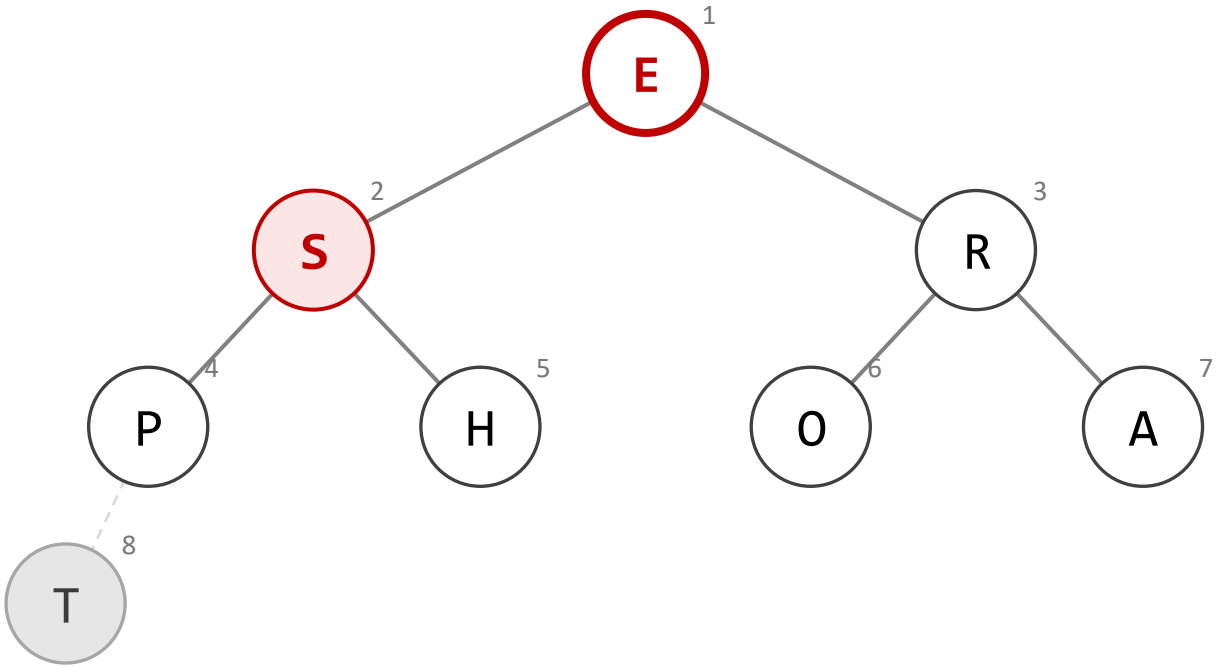


the item that's sinking

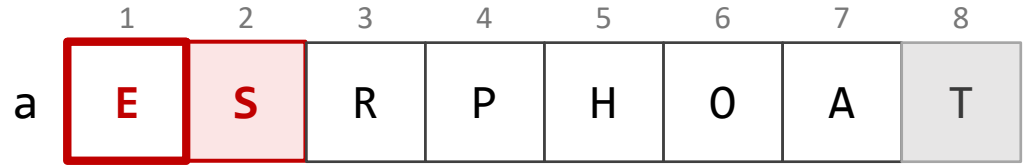
its larger child

sorted: no longer in the heap

Phase 2: sortdown



sink(a, 1, 7). E is at position 1. Its children are positions 2 and 3: S and R. The larger is **S**. Is E less than S? **Yes**: exchange them.

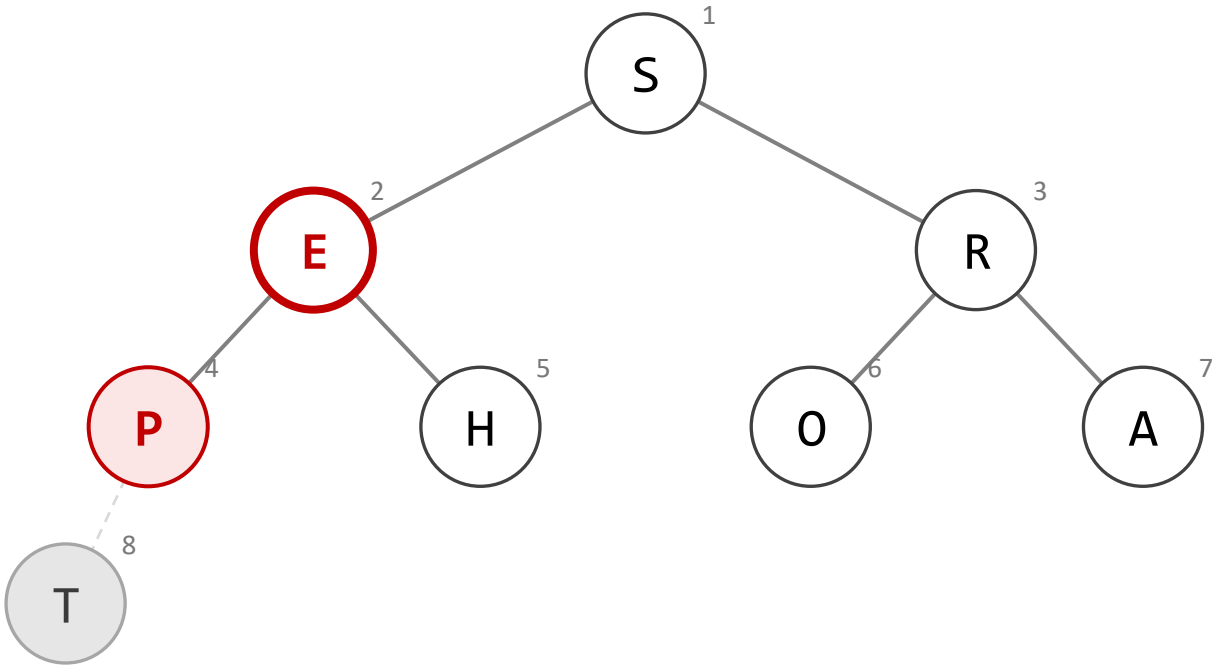


the item that's sinking

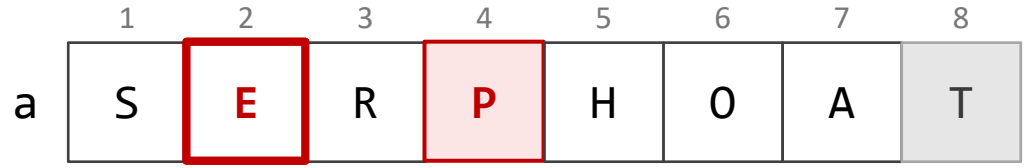
its larger child

sorted: no longer in the heap

Phase 2: sortdown



sink(a, 1, 7). E is at position 2. Its children are positions 4 and 5: P and H. The larger is **P**. Is E less than P? **Yes**: exchange them.

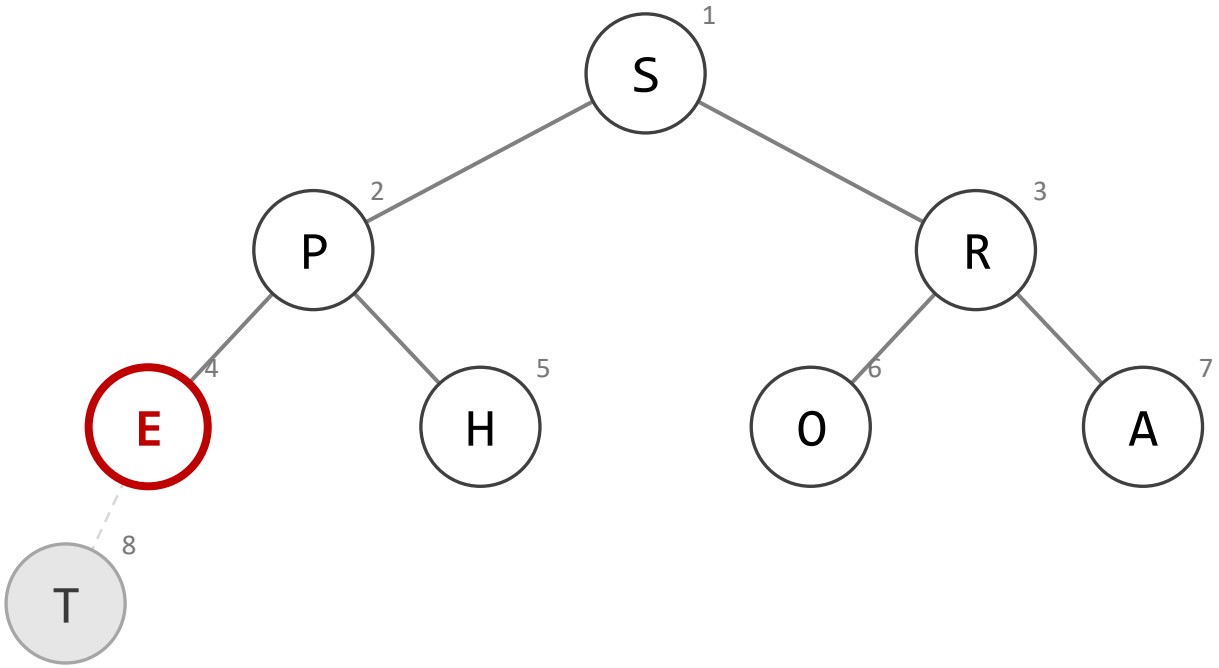


the item that's sinking

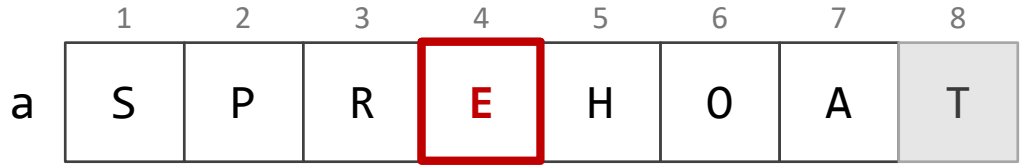
its larger child

sorted: no longer in the heap

Phase 2: sortdown



sink(a, 1, 7). E is at position 4, which has no children in the heap (8 > 7). It stops.

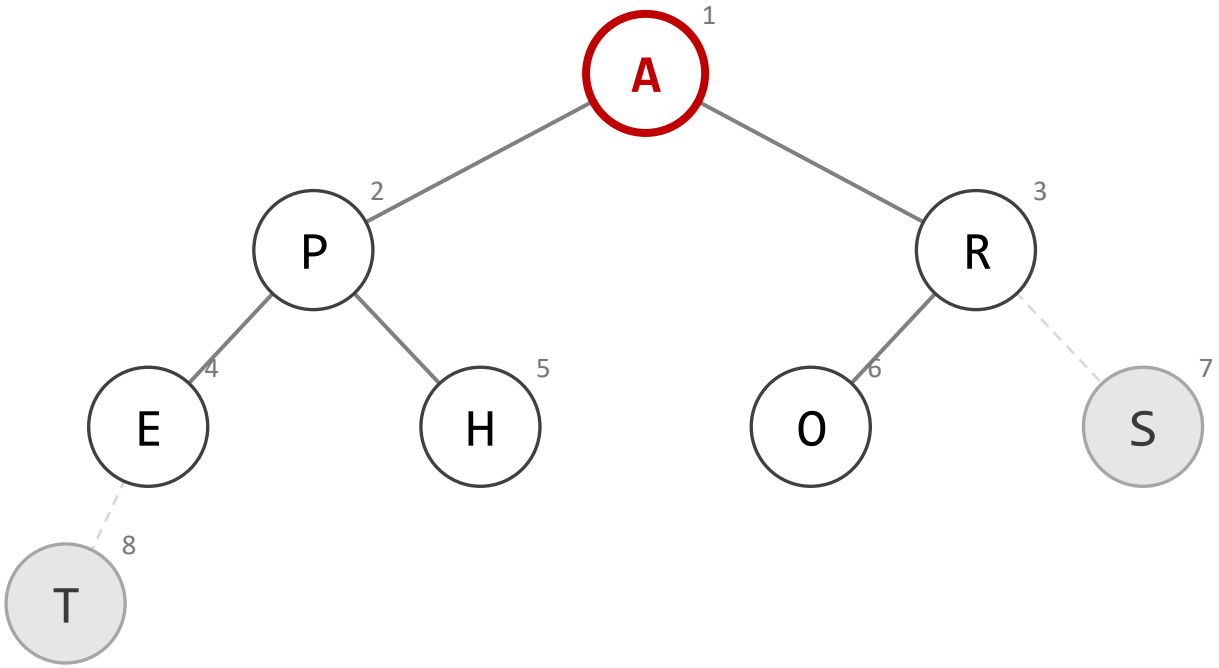


the item that's sinking

its larger child

sorted: no longer in the heap

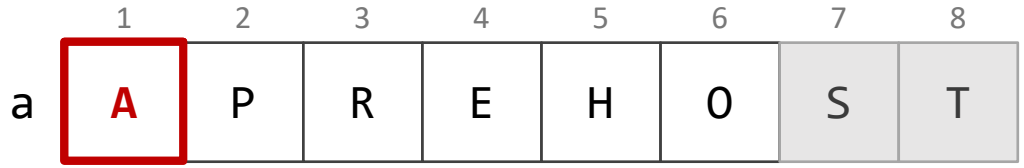
Phase 2: sortdown

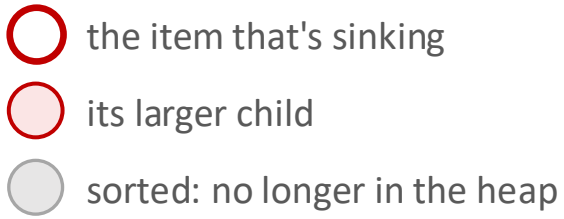


exch(a, 1, 7): the largest, **S**, swaps with the last item in the heap, **A**.

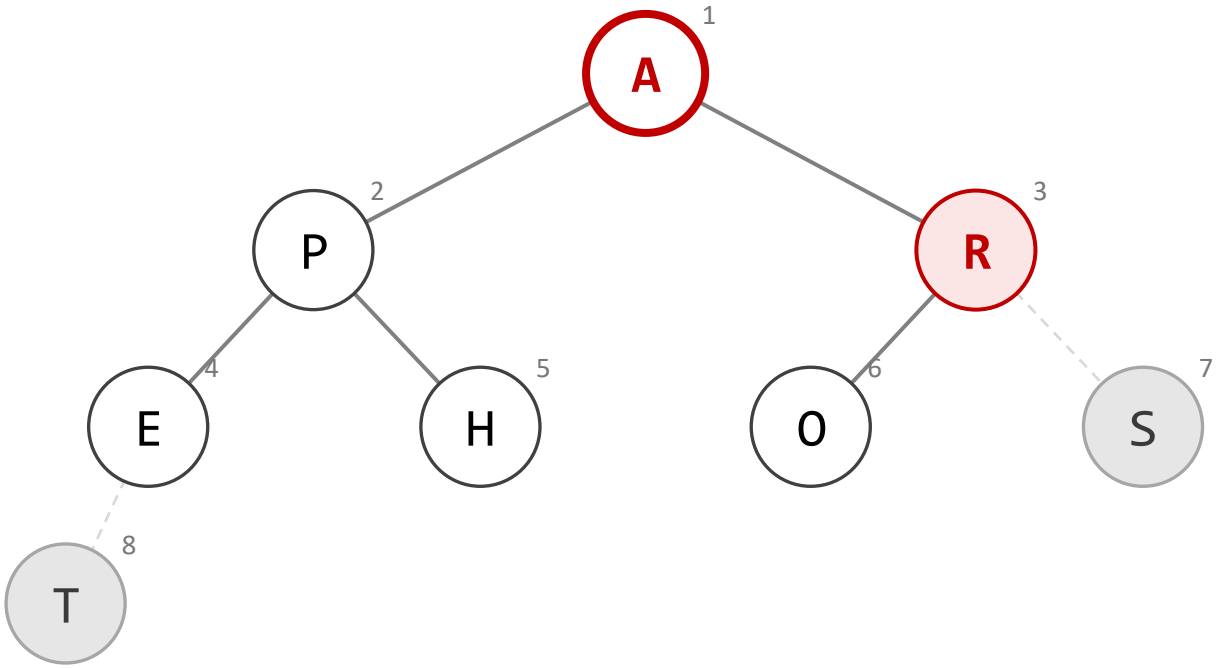
n--: S is in its final spot (gray). The heap has **6** items.

Now sink **A** from the root.

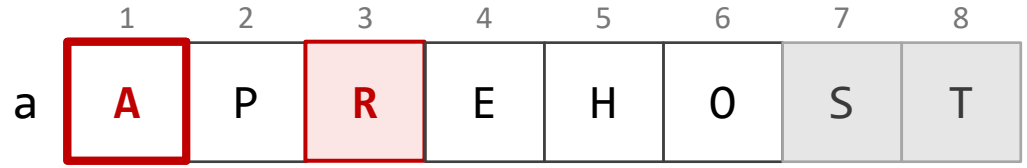




Phase 2: sortdown



sink(a, 1, 6). A is at position 1. Its children are positions 2 and 3: P and R. The larger is **R**. Is A less than R? **Yes**: exchange them.

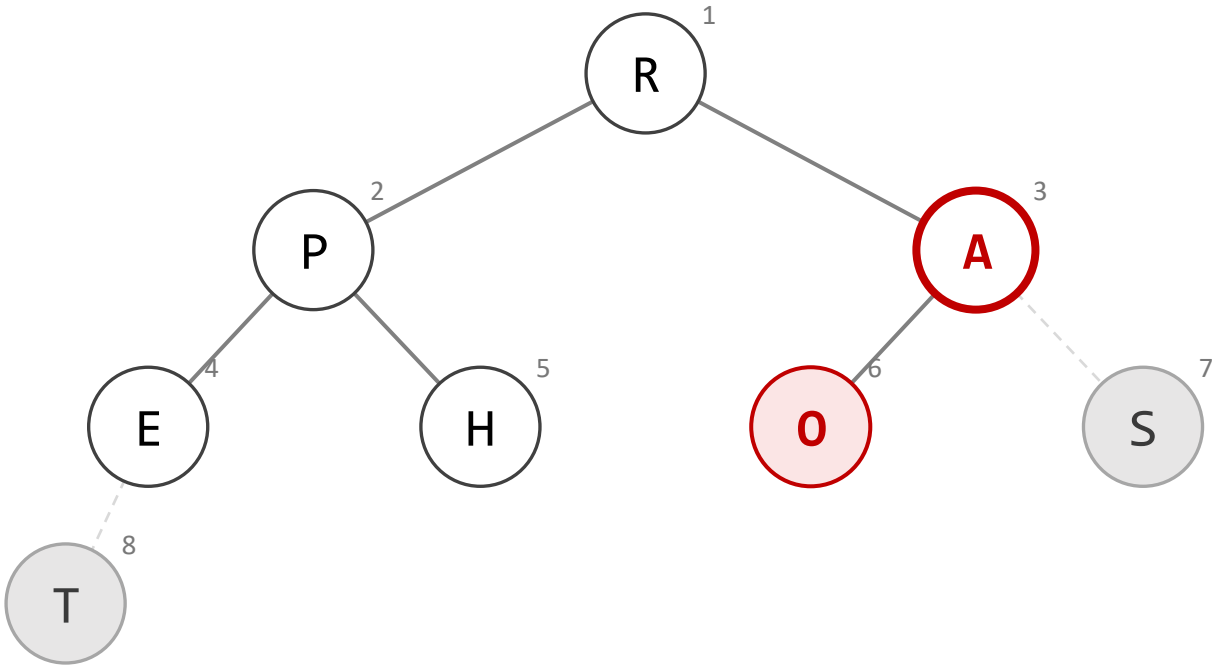


the item that's sinking

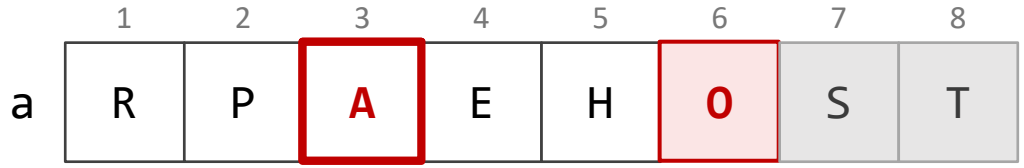
its larger child

sorted: no longer in the heap

Phase 2: sortdown



sink(a, 1, 6). A is at position 3. Its only child in the heap is position 6: O. Is A less than O? **Yes**: exchange them.

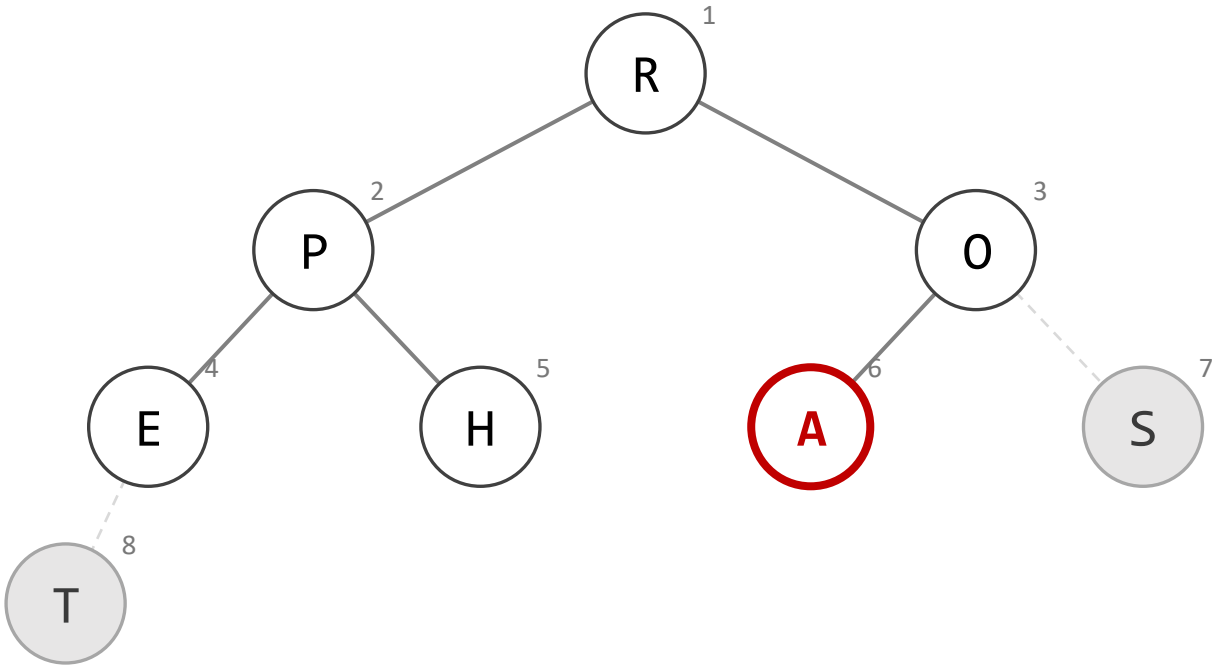


the item that's sinking

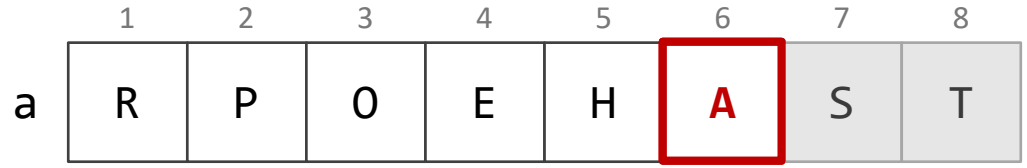
its larger child

sorted: no longer in the heap

Phase 2: sortdown



sink(a, 1, 6). A is at position 6, which has no children in the heap (12 > 6). It stops.

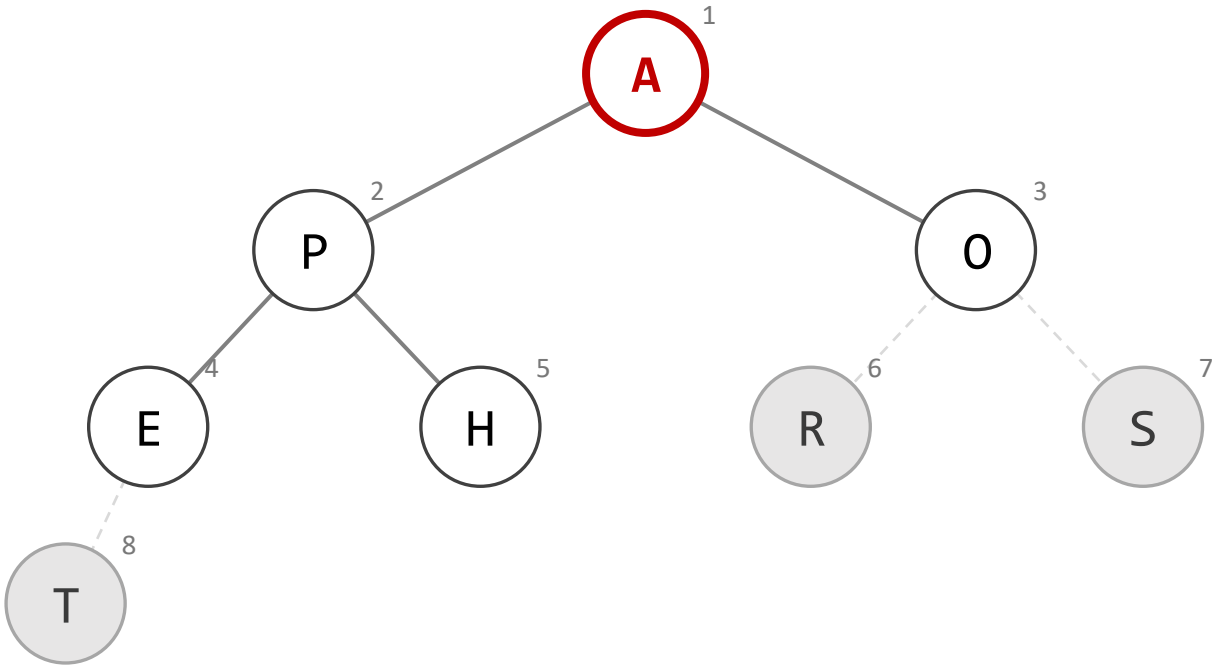


the item that's sinking

its larger child

sorted: no longer in the heap

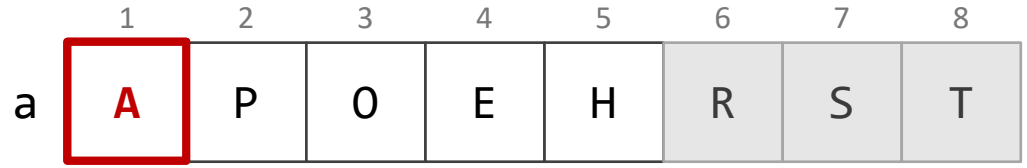
Phase 2: sortdown



exch(a, 1, 6): the largest, **R**, swaps with the last item in the heap, **A**.

n--: R is in its final spot (gray). The heap has **5** items.

Now sink **A** from the root.

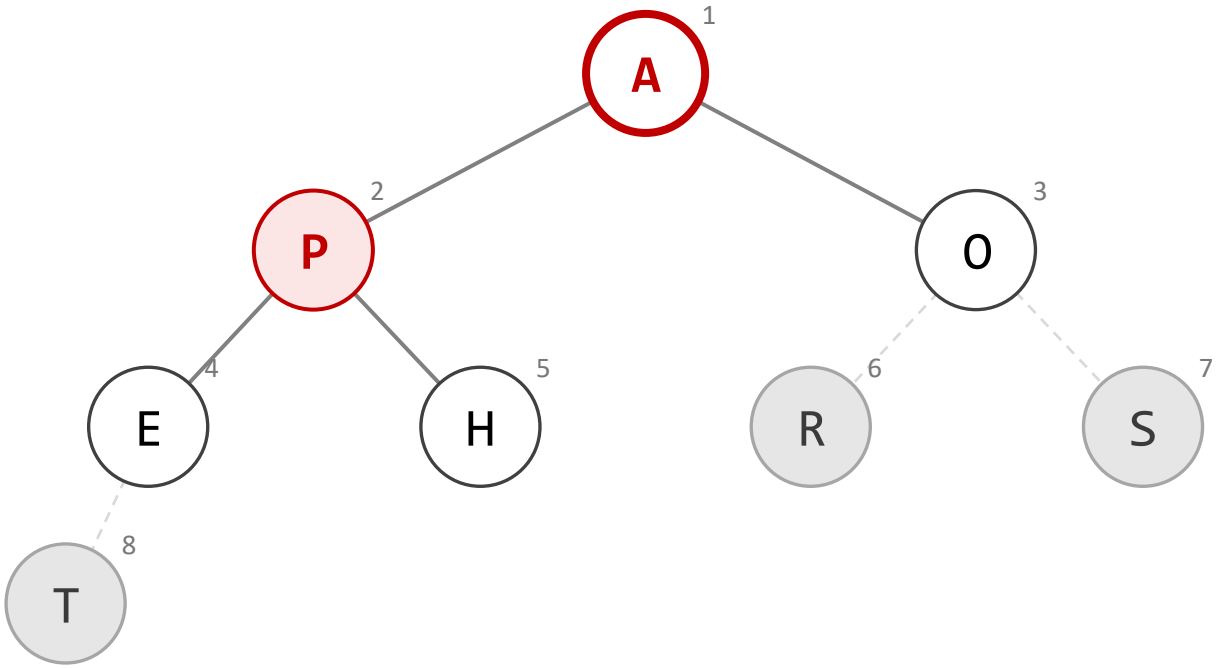


the item that's sinking

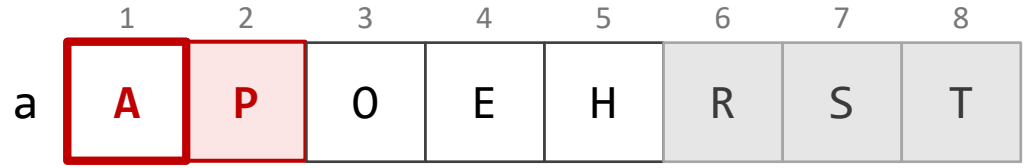
its larger child

sorted: no longer in the heap

Phase 2: sortdown



sink(a, 1, 5). A is at position 1. Its children are positions 2 and 3: P and O. The larger is **P**. Is A less than P? **Yes**: exchange them.

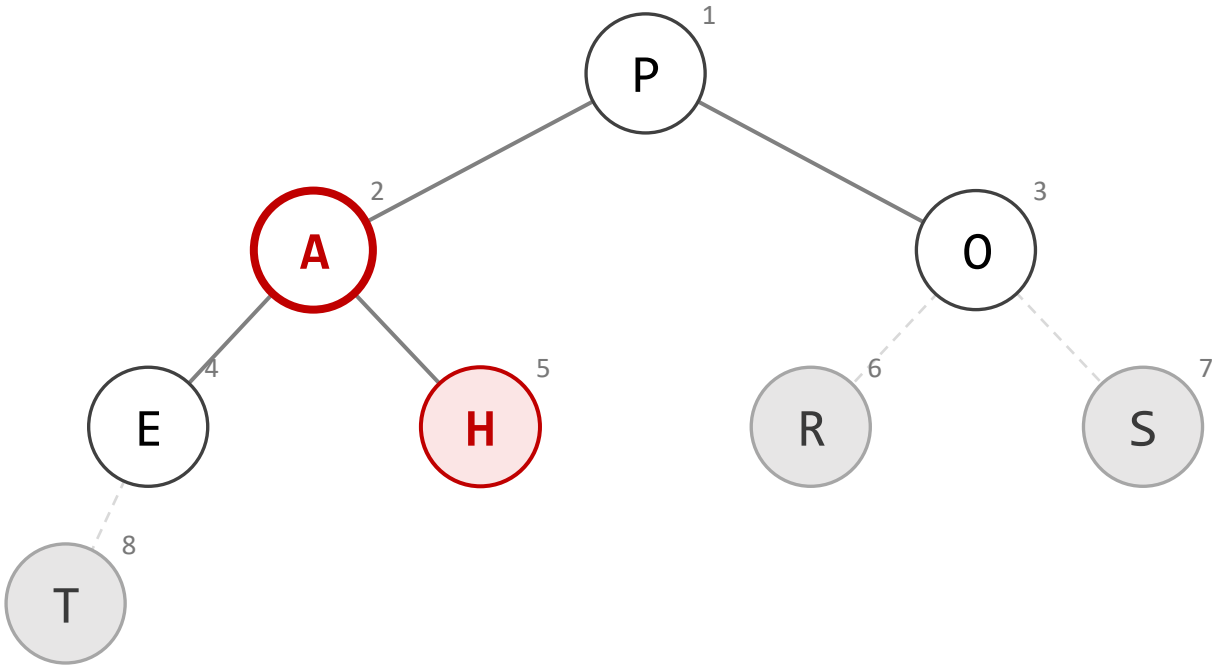


the item that's sinking

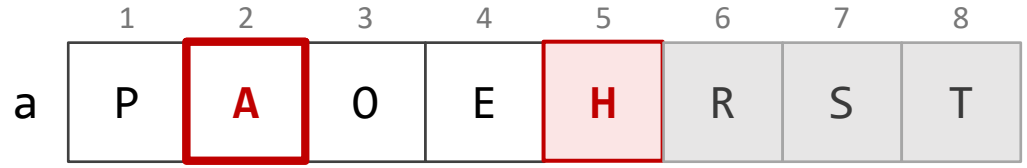
its larger child

sorted: no longer in the heap

Phase 2: sortdown



sink(a, 1, 5). A is at position 2. Its children are positions 4 and 5: E and H. The larger is **H**. Is A less than H? **Yes**: exchange them.

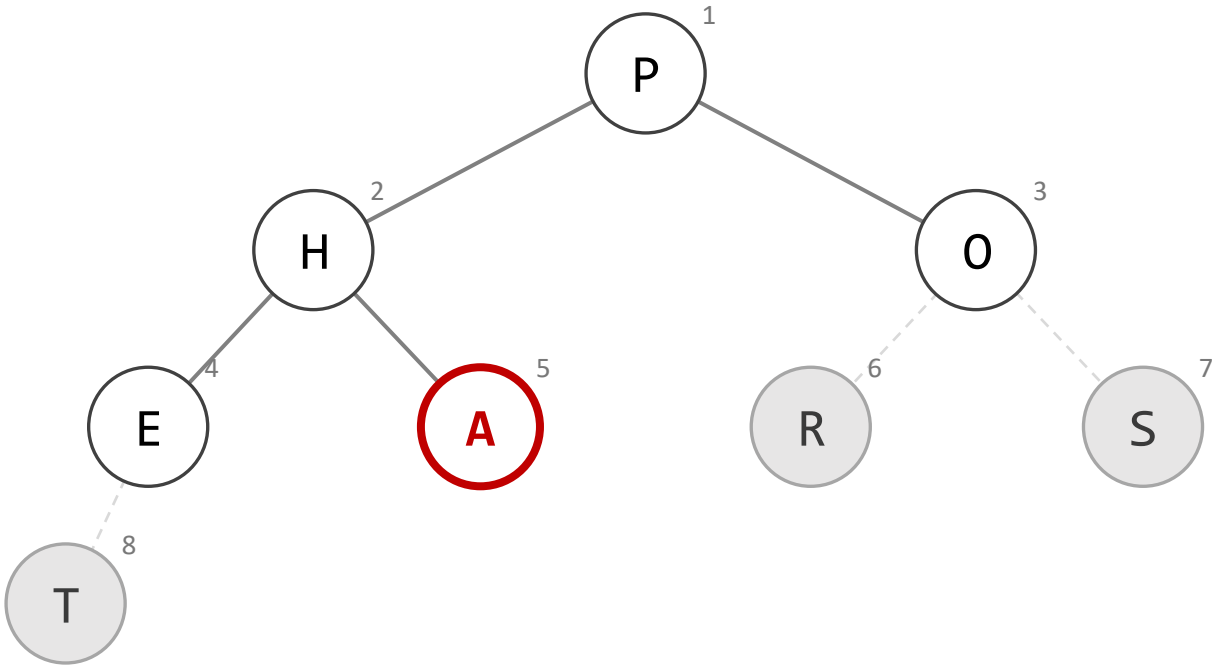


the item that's sinking

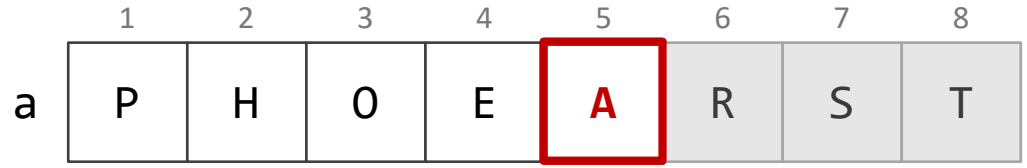
its larger child

sorted: no longer in the heap

Phase 2: sortdown



sink(a, 1, 5). A is at position 5, which has no children in the heap (10 > 5). It stops.

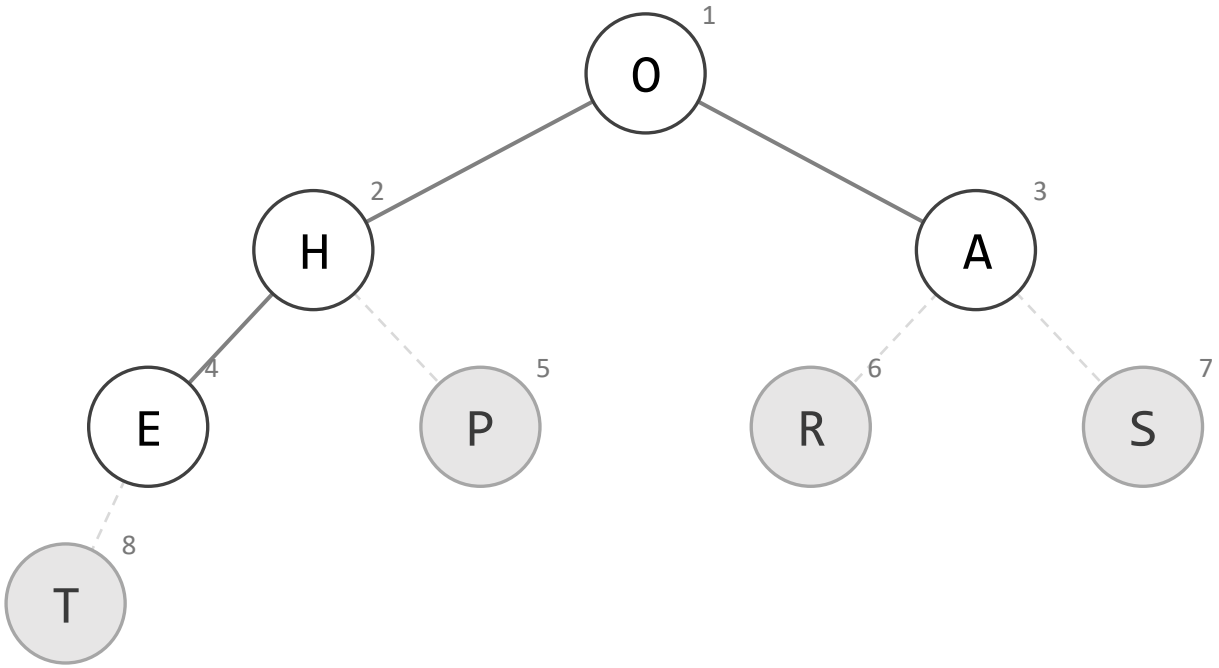


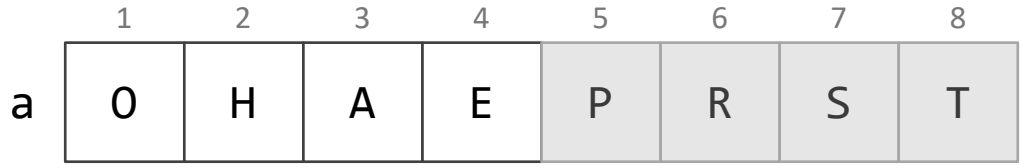
the item that's sinking

its larger child

sorted: no longer in the heap

Phase 2: sortdown





Each step is the same three moves, so from here on there's one slide per step.

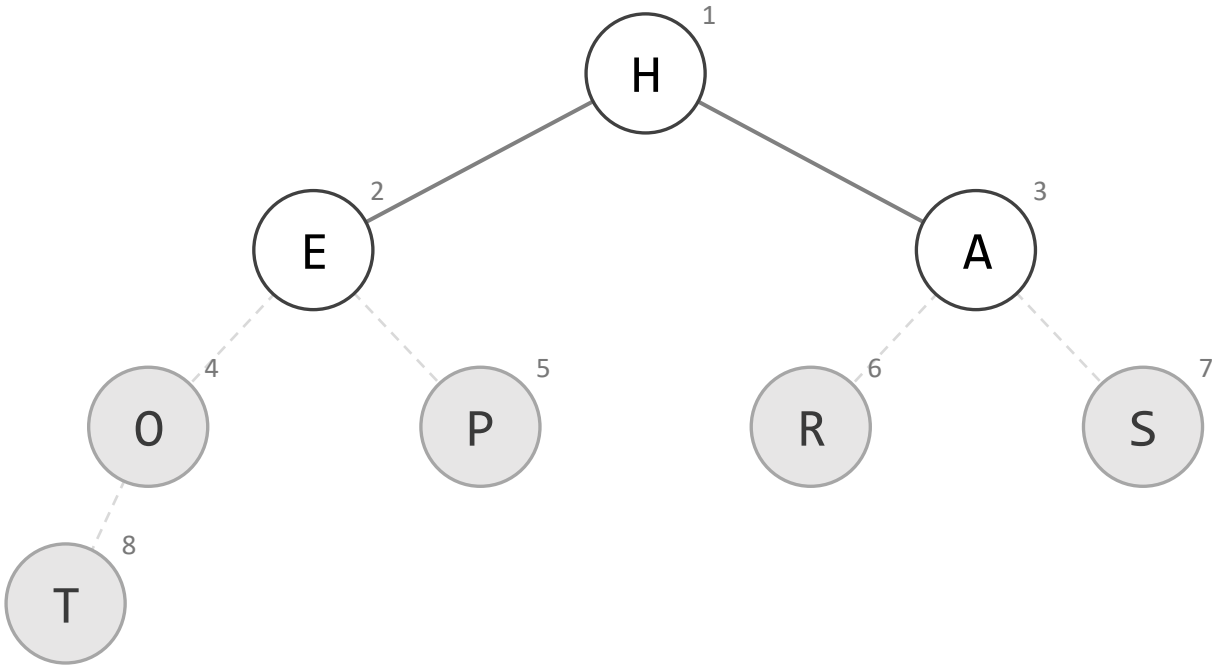
exch(a, 1, 5): P goes to position 5. Then **sink** A: 1 exchange.

the item that's sinking

its larger child

sorted: no longer in the heap

Phase 2: sortdown



exch(a, 1, 4): O goes to position 4. Then **sink** E: 1 exchange.

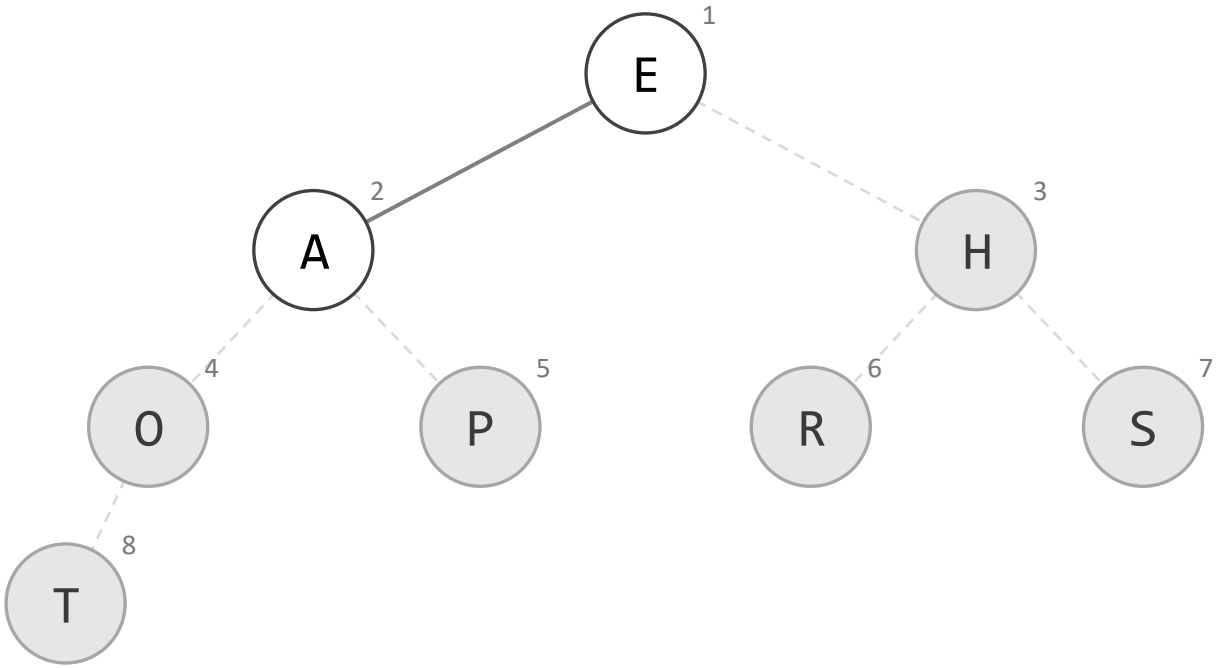


the item that's sinking

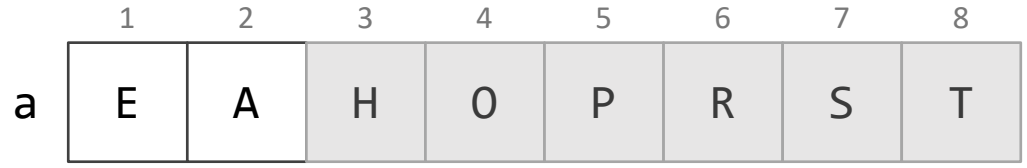
its larger child

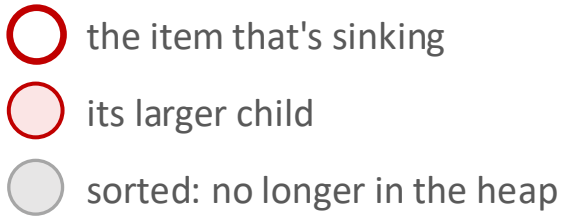
sorted: no longer in the heap

Phase 2: sortdown

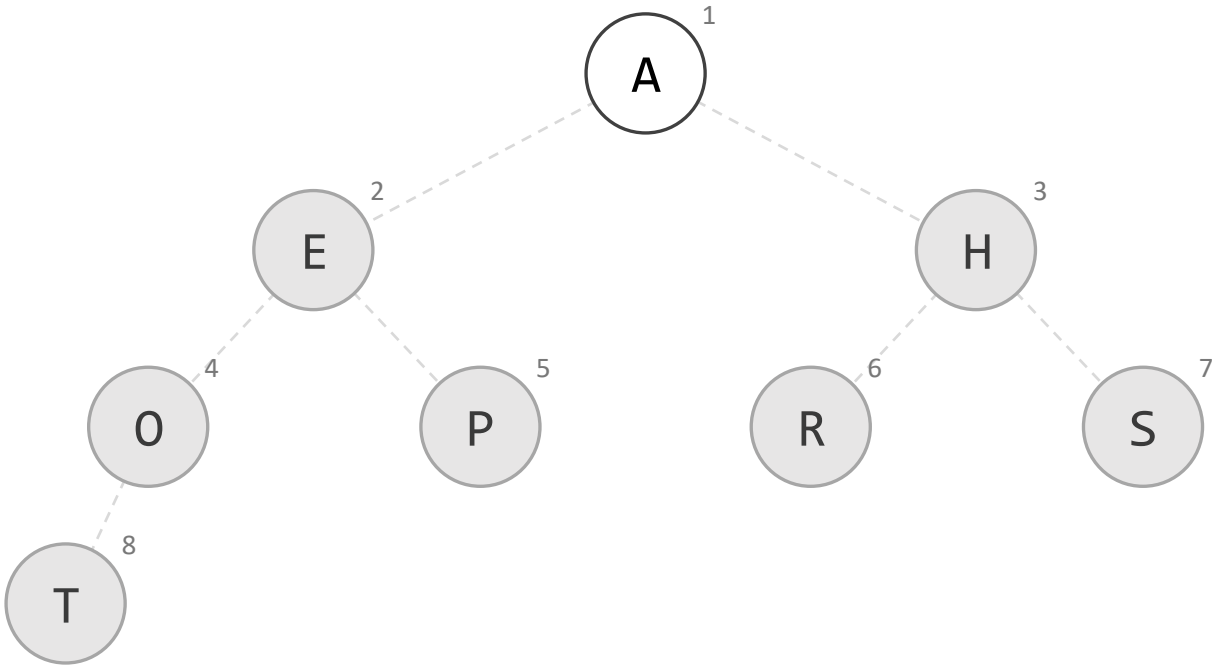


exch(a, 1, 3): H goes to position 3. Then **sink** A: 1 exchange.





Phase 2: sortdown



exch(a, 1, 2): E goes to position 2. Then **sink** A: 0 exchanges.

	1	2	3	4	5	6	7	8
a	A	E	H	O	P	R	S	T

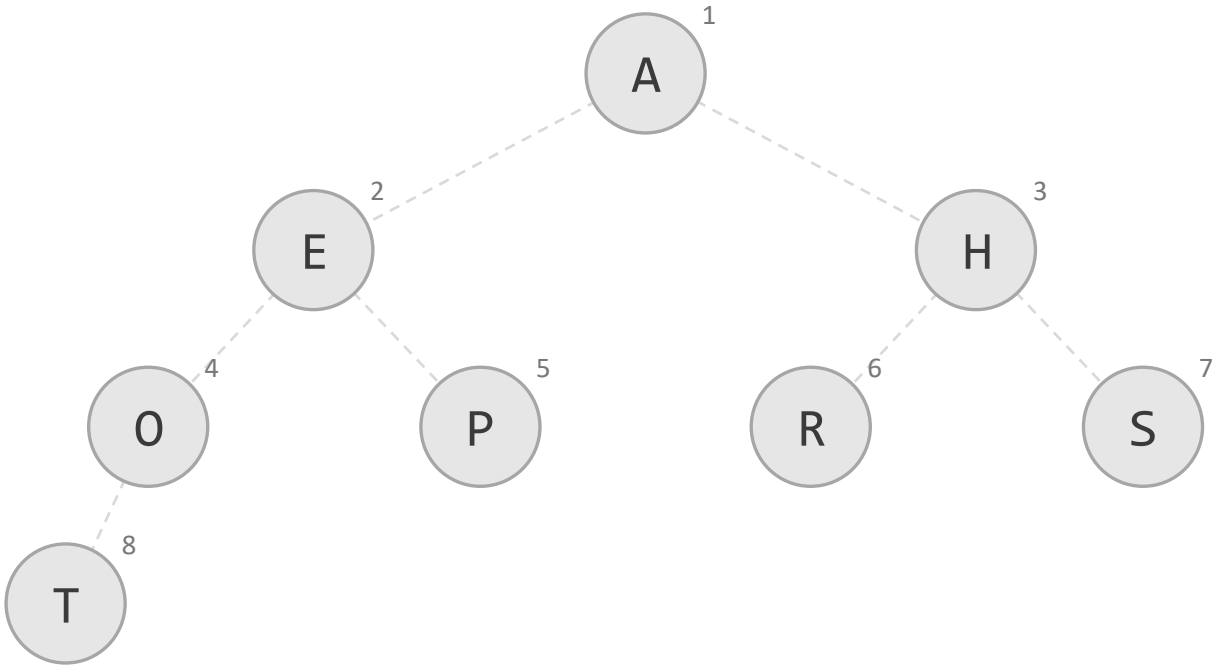
the item that's sinking

its larger child

sorted: no longer in the heap

Phase 2: sortdown

done





Done. The heap is down to one item, **A**, which is already in place.

The array is sorted: **A E H O P R S T**.

We never needed a second array.

Heapsort: the code

```
1  public static void sort(Comparable[] a) {
2      int n = a.length;
3      for (int k = n / 2; k >= 1; k--) {   // phase 1
4          sink(a, k, n);
5      }
6      while (n > 1) {                      // phase 2
7          exch(a, 1, n);
8          n--;
9          sink(a, 1, n);
10     }
11 }
```

```
1  private static void sink(Comparable[] a, int k, int n) {
2      while (2 * k <= n) {
3          int j = 2 * k;
4          if (j < n && less(a, j, j + 1)) j++;
5          if (!less(a, k, j)) break;
6          exch(a, k, j);
7          k = j;
8      }
9  }
```

Lines 3–5: **phase 1**, build a heap.

Lines 6–10: **phase 2**, the sortdown.

`sink` is the same as in MaxPQ, but it takes the array and the heap size **n**, because n shrinks in phase 2.

less and exch convert from position to index

```
1  // Heap position i is stored in a[i - 1].
2  private static boolean less(Comparable[] a, int i, int j) {
3      return a[i - 1].compareTo(a[j - 1]) < 0;
4  }
5  private static void exch(Comparable[] a, int i, int j) {
6      Comparable t = a[i - 1];
7      a[i - 1] = a[j - 1];
8      a[j - 1] = t;
9  }
```

`less` and `exch` take **positions** 1 to n, and subtract 1 when they touch the array.

So `less(a, 1, 2)` compares `a[0]` with `a[1]`.

This is the **only** place the – 1 appears. If heapsort crashes with an index out of bounds, or leaves the first item out of place, check these two methods first.

What heapsort costs

Compares: fewer than **2n lg n + 2n**, even in the **worst case**.

Memory: heapsort sorts **in place**: it doesn't need a second array, unlike mergesort.

Not stable. Sort **B_1 B_2 A**, and you get **A B_2 B_1**: the two equal Bs switched order.

Usually slower than quicksort in practice. It jumps around the array, and computers are fastest when they read memory in order.

Where it's used: the Linux kernel's `sort()`, and as the backup plan inside C++'s `std::sort` when quicksort is going badly.

The sorts we've seen

sort	compares (typical)	worst case	extra memory	stable?
insertion	~$n^2/4$	~$n^2/2$	none	yes
mergesort	~n lg n	~n lg n	an array of n	yes
quicksort	~1.39 n lg n	~$n^2/2$	about lg n (recursion)	no
heapsort	**~2n lg n**	**~2n lg n**	**none**	**no**

None of these sorts is the best choice in every column, so real libraries often combine them. For example, a library might use quicksort, switch to insertion sort for small pieces, and switch to heapsort if quicksort starts going badly.